

Real Projects

Putting everything together — eight complete, buildable AI project blueprints: chatbot, coding assistant, study assistant, research assistant, chat with PDF, documentation generator, email assistant, and workflow automation.

DIFFICULTY LEVEL

Intermediate-Advanced

ESTIMATED READING TIME

55-70 minutes

ESTIMATED PRACTICE TIME

3-6 hours (project-dependent)

PREREQUISITES

Modules 01-13 (the entire bucket)

LEARNING OUTCOMES

By the end of this module, you will be able to: map any of these 8 project types to the specific modules and techniques they require; scope and plan a real AI project from idea to architecture; and build at least one complete, working AI-powered project end to end.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	AI Chatbot
3.2	AI Coding Assistant
3.3	AI Study Assistant
3.4	AI Research Assistant
3.5	Chat with PDF
3.6	AI Documentation Generator
3.7	AI Email Assistant
3.8	AI Workflow Automation
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project (Capstone)
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

This is the capstone module. Every technique from Modules 01-13 — prompting, structured outputs, context engineering, tool use, RAG, agents, optimization, safety, model/API choice, and workflows — comes together here as 8 concrete, buildable project blueprints.

Why is it important?

Knowledge that's never been assembled into a real, working thing stays abstract. This module is where the bucket's individual modules become a genuine skill — the ability to look at a project idea and know exactly which techniques to reach for.

13 Modules of Skills



Assembled Together



One Real, Working Project

Why should AI Engineers learn it?

Every project below maps to real product categories built and shipped by real companies. Building even one of these end to end is the single highest-leverage thing you can do to consolidate everything in this course — and gives you something concrete for a portfolio.

Where is it used in the real world?

Each project blueprint below corresponds to an actual, common category of AI product in production today — you're not building toy exercises, you're building smaller versions of real things.

Why is it relevant today?

The gap between "understands AI concepts" and "can ship an AI feature" is exactly what this module closes — and it's the gap that matters most for actually getting hired or shipping a product in 2026's AI engineering job market.

Note

You don't need to build all 8. Pick 1-2 that genuinely interest you or match a real need you have, and build them properly — a single well-built project teaches more than 8 rushed ones.

2. Before You Begin

RECAP

Everything. This module assumes working familiarity with Modules 01-13. If a technique mentioned below feels unfamiliar, that module number is noted so you can revisit it.



Tip

Before writing any code, sketch your chosen project's architecture on paper first: what modules/techniques does it need, in what order? A 10-minute sketch saves hours of restructuring later.

3. Main Lesson

Each project follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Key Modules Used** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 AI Chatbot

DEFINITION

A conversational assistant that maintains context across a multi-turn conversation, answering questions and helping with tasks in natural language.

SIMPLE EXPLANATION

The foundational project type — a system prompt defining persona/scope, conversation history management, and a clean chat interface, without necessarily needing tools, RAG, or agentic autonomy.

 *The "hello world" of AI products — simple in concept, but every real-world consideration (context limits, persona consistency, cost) shows up even at this basic level.*

EVERYDAY EXAMPLE

A customer-facing FAQ assistant, a personal journaling companion, or a niche-topic expert bot (e.g., only answers cooking questions).

KEY MODULES USED

Module 02 (persona/system prompt), Module 05 (conversation history management), Module 09 (cost/latency tuning for a chat product).

COMMON MISTAKES

Sending unbounded conversation history (Module 05, 3.5) without a management strategy, causing costs and context usage to grow unchecked.

BEST PRACTICES

Define a clear, narrow persona and scope upfront; implement a sliding-window or summarization strategy for history from the start, not as an afterthought.

SUMMARY

The AI chatbot is the essential starting project — deceptively simple, but where every core context-management skill first gets exercised in a real system.

3.2 AI Coding Assistant

DEFINITION

A tool that helps write, review, explain, or debug code — ranging from simple prompt-based helpers to agentic tools that read and modify a real codebase.

SIMPLE EXPLANATION

At a basic level, this can be a specialized chatbot (Section 3.1) with a code-focused system prompt; at a more advanced level, it becomes a true agent (Module 08) with file-read/write and test-running tools (Module 06).

 *Like the difference between a colleague who reviews code you show them (basic assistant) and one who's given repo access to actually make the fix themselves (agentic version).*

EVERYDAY EXAMPLE

A CLI tool that takes a bug description, reads the relevant file, proposes a fix, and (optionally) applies it after your confirmation.

KEY MODULES USED

Module 06 (file read/write, run-tests tools), Module 08 (agent loop for multi-step fixes), Module 10 (guardrails/confirmation before applying changes).

COMMON MISTAKES

Auto-applying code changes without any review step (Module 10, 3.6) — even a simple confirmation prompt meaningfully reduces risk.

BEST PRACTICES

Start with a read-only version (suggests changes, doesn't apply them) before adding write access; always run tests after any agentic change.

SUMMARY

The AI coding assistant scales from a simple specialized chatbot to a full agentic system — a great project for exercising Module 06 and 08 together.

🛠️ 3.3 AI Study Assistant

DEFINITION

A tool that helps a learner understand material — explaining concepts, generating practice questions, and adapting to what the learner already knows.

SIMPLE EXPLANATION

Combines persona prompting (a patient, encouraging tutor voice), few-shot/chain-of-thought techniques (Module 03) for clear explanations, and often RAG (Module 07) grounded in the learner's actual course material.

💡 *Like a private tutor who has actually read your specific textbook, rather than one who only knows the subject in general terms — grounding matters here.*

EVERYDAY EXAMPLE

A tool that takes a student's lecture notes, generates practice quiz questions from them, and explains wrong answers with tailored analogies.

KEY MODULES USED

Module 02 (tutor persona), Module 03 (step-by-step explanations), Module 07 (RAG grounded in the specific course material).

COMMON MISTAKES

Letting the assistant answer purely from general trained knowledge instead of the learner's actual course material, risking mismatched terminology or scope.

BEST PRACTICES

Ground explanations in the learner's own uploaded material (RAG) wherever possible; use chain-of-thought (Module 03, 3.5) explicitly when explaining multi-step problems.

SUMMARY

The AI study assistant blends persona, careful explanation techniques, and grounding — a strong project for practicing RAG in a genuinely personal context.

🔍 3.4 AI Research Assistant

DEFINITION

An agent that searches, reads, synthesizes, and cites information across multiple sources to answer a research question or produce a report.

SIMPLE EXPLANATION

This is a genuine multi-step agent (Module 08) using a search tool (Module 06), often combined with re-ranking (Module 07, 3.6) for source quality, and needing careful

hallucination handling (Module 10, 3.3) since factual accuracy matters enormously.

💡 *Like hiring a research assistant who doesn't just Google one thing, but iteratively searches, reads, cross-checks, and compiles findings into a coherent, cited report.*

EVERYDAY EXAMPLE

A tool that takes "summarize the current state of solid-state battery research" and autonomously searches, reads several sources, and produces a cited summary.

KEY MODULES USED

Module 08 (agent loop with planning), Module 06 (search tool), Module 10 (hallucination handling, source grounding).

COMMON MISTAKES

Not requiring citations tied to actual retrieved sources, letting the agent state plausible-sounding but unverified claims.

BEST PRACTICES

Require every claim to be traceable to a specific retrieved source; set a bounded step limit (Module 08, 3.6) so research doesn't run indefinitely.

SUMMARY

The AI research assistant is one of the most demanding projects here — a real test of agent design, tool use, and hallucination discipline together.

📄 3.5 Chat with PDF

DEFINITION

A tool that lets a user upload a document and ask natural-language questions about it, with answers grounded strictly in that document's content.

SIMPLE EXPLANATION

This is a focused, single-document RAG system (Module 07): chunk the PDF, embed it, retrieve relevant sections per question, and answer only from retrieved content — a clean, well-scoped project to practice the full RAG pipeline end to end.

💡 *Like handing someone a specific document and asking them to only answer based on what's actually written in it, rather than from their general outside knowledge.*

EVERYDAY EXAMPLE

Uploading a rental contract and asking "what's the penalty for breaking the lease early?" with the answer grounded in and citing the specific contract clause.

KEY MODULES USED

Module 07 (the entire RAG pipeline — chunking, embeddings, retrieval), Module 05 (context injection with delimiters), Module 10 (explicit "answer only from this document" grounding instruction).

COMMON MISTAKES

Letting the model answer from general knowledge when the document doesn't actually contain the answer, instead of clearly saying so.

BEST PRACTICES

Explicitly instruct the model to say when an answer isn't in the document; show which section/page an answer was drawn from for user trust.

SUMMARY

Chat with PDF is the cleanest, most self-contained project for mastering the full RAG pipeline from Module 07.

📖 3.6 AI Documentation Generator

DEFINITION

A tool that generates structured documentation (READMEs, API docs, code comments) automatically from a codebase or a set of inputs.

SIMPLE EXPLANATION

This project leans heavily on structured output (Module 04) — generating consistently-formatted Markdown or structured docs — often combined with prompt chaining (Module 03, 3.10): first extract structure, then generate content, then format.

💡 *Like a technical writer who reads your code and produces a consistent, well-organized manual — the value is as much in the consistent structure as the content itself.*

EVERYDAY EXAMPLE

A tool that reads a Python module's functions and docstrings and generates a formatted API reference page automatically.

KEY MODULES USED

Module 04 (structured/Markdown output), Module 03 (prompt chaining: extract → generate → format), Module 09 (template versioning for consistent doc style).

COMMON MISTAKES

Generating documentation with an inconsistent structure across different runs, undermining the whole point of automated docs.

BEST PRACTICES

Use a strict template (Module 02, 3.9) so output structure stays consistent; validate generated Markdown/structure before publishing.

SUMMARY

The AI documentation generator is a strong project for practicing structured output and prompt templates in a genuinely useful tool.

📖 3.7 AI Email Assistant

DEFINITION

A tool that helps draft, summarize, or triage emails — ranging from a simple draft-generator to an agent that reads an inbox and takes actions.

SIMPLE EXPLANATION

At a basic level, this is prompt templates (Module 02, 3.9) for common email types; at an advanced level, it's a tool-using agent (Module 06, 08) reading an inbox via an email API and drafting or even sending replies, with strong guardrails (Module 10) given the sensitivity of email.

💡 *Like a assistant who can draft your replies for your review (safe, basic version) versus one trusted to actually send emails on your behalf (advanced, higher-stakes version).*

EVERYDAY EXAMPLE

A tool that reads a long email thread and drafts a concise reply matching your usual tone, for you to review and send.

KEY MODULES USED

Module 02 (persona matching your writing tone), Module 06 (email API tool integration), Module 10 (mandatory human confirmation before sending anything).

COMMON MISTAKES

Giving the assistant send-access without a confirmation step (Module 10, 3.6) — email is

exactly the kind of high-stakes action that needs a human checkpoint.

BEST PRACTICES

Start with draft-only mode; require explicit user approval before any email is actually sent, no matter how confident the system seems.

SUMMARY

The AI email assistant is a great project for practicing tool use with real consequences — and a clear case for guardrails done right.

⚙️ 3.8 AI Workflow Automation

DEFINITION

A system combining AI decision-making with multi-step automated actions across tools/services, either as custom code or built on a no-code platform (Module 13).

SIMPLE EXPLANATION

This project type applies function calling (Module 06) and often prompt chaining (Module 03, 3.10) to real business processes — either coded directly with an API, or built visually using n8n/Make/Zapier (Module 13).

💡 *Like setting up an assembly line where an AI step makes a judgment call (classify, summarize, decide) at one or more points along an otherwise automated process.*

EVERYDAY EXAMPLE

New support ticket arrives → AI classifies urgency and topic → routes to the right team → drafts a first-response suggestion — all automated except final human send.

KEY MODULES USED

Module 06 (tool/API integration), Module 13 (no-code platform option), Module 09 (cost optimization at automation scale), Module 10 (validation at each automated step).

COMMON MISTAKES

Automating a decision step with no validation (Module 04, 10) on the AI's output before it flows into the next automated action.

BEST PRACTICES

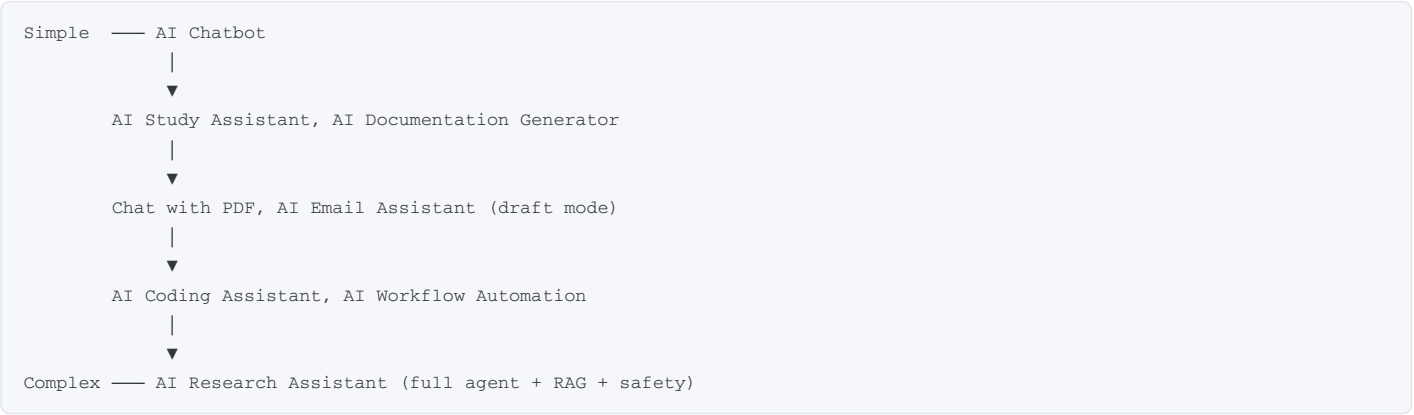
Validate AI output at each step before it triggers the next automated action; start with human-in-the-loop checkpoints, remove them only once the workflow has proven reliable.

SUMMARY

AI workflow automation ties together tool use, structured output, and either code or no-code platforms into a real, running business process.

4. Visual Learning

🏗️Project Complexity Ladder



🔑Which Modules Each Project Leans On Most

Real Projects

- **Heavy on RAG (Module 07)** *Chat with PDF, Study Assistant*
- **Heavy on Agents (Module 08)** *Coding Assistant, Research Assistant*
- **Heavy on Structured Output (Module 04)** *Documentation Generator*
- **Heavy on Tool Use + Safety (Modules 06, 10)** *Email Assistant, Workflow Automation*

📊Project-to-Module Map

Project	Core Modules
AI Chatbot	02, 05
AI Coding Assistant	06, 08, 10
AI Study Assistant	02, 03, 07
AI Research Assistant	06, 07, 08, 10
Chat with PDF	05, 07, 10
AI Documentation Generator	03, 04, 09
AI Email Assistant	02, 06, 10
AI Workflow Automation	06, 09, 10, 13

5. Real World Applications

Project Type	Real Product Examples
AI Chatbot	Customer support widgets, brand assistants
AI Coding Assistant	Claude Code, GitHub Copilot, Cursor (Module 13)
Chat with PDF	Document Q&A tools for legal, medical, and research documents
AI Research Assistant	Deep-research features in consumer AI products
AI Workflow Automation	AI-augmented n8n/Make/Zapier flows (Module 13) in business ops

6. Practical Examples

Beginner: Picking your project

Review all 8 projects and pick the one that solves a real need you personally have. Write 2-3 sentences on why that one, specifically.

Beginner: Module mapping

For your chosen project, list which modules from this course you'll need to lean on most, using Section 4's table as a starting point.

Intermediate: Architecture sketch

Sketch your chosen project's full architecture on paper: inputs, processing steps, external calls (APIs/tools), and outputs.

Advanced: Scoping a v1 vs. v2

Define a minimal "v1" version of your project (what's the smallest version that's genuinely useful?) and a "v2" with 2-3 planned enhancements — practicing the incremental scoping real engineers use.

7. Code Examples

🐍Python — Minimal "Chat with PDF" skeleton (combining Modules 04, 05, 07, 10)

```
# pip install anthropic numpy pypdf
import anthropic
from pypdf import PdfReader

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# --- Module 07: Chunking ---
def load_and_chunk_pdf(path):
    reader = PdfReader(path)
    text = "\n".join(page.extract_text() for page in reader.pages)
    # simplified: split by paragraph; a real system would be smarter about boundaries
    return [p.strip() for p in text.split("\n\n") if p.strip()]

# --- Module 07: Embedding + retrieval (pseudo, plug in a real embedding model) ---
def retrieve(query, chunks, top_k=3):
    # In real code: embed query + chunks, rank by cosine similarity (Module 07, 7)
    return chunks[:top_k] # placeholder

# --- Module 05 + 10: Safe, delimited context injection with grounding instruction ---
def answer_question(query, chunks):
    context = "\n---\n".join(retrieve(query, chunks))
    prompt = f"""<document_excerpts>
{context}
</document_excerpts>

Answer ONLY using the excerpts above. If the answer isn't there,
say "This isn't covered in the document." Question: {query}"""

    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

chunks = load_and_chunk_pdf("contract.pdf")
print(answer_question("What's the penalty for early termination?", chunks))
```

📄JavaScript — Module-mapping helper for planning your own project

```
// A tiny planning tool: given a project type, suggests which modules to review
const projectModuleMap = {
  chatbot: [2, 5],
  codingAssistant: [6, 8, 10],
  studyAssistant: [2, 3, 7],
  researchAssistant: [6, 7, 8, 10],
  chatWithPdf: [5, 7, 10],
  docGenerator: [3, 4, 9],
  emailAssistant: [2, 6, 10],
  workflowAutomation: [6, 9, 10, 13],
};

function suggestModulesToReview(projectType) {
  const modules = projectModuleMap[projectType];
  if (!modules) return "Unknown project type";
  return `Before building, review Modules: ${modules.join(", ")}`;
}

console.log(suggestModulesToReview("chatWithPdf"));
// → "Before building, review Modules: 5, 7, 10"
```

✓ Tip

The Chat with PDF skeleton above deliberately uses placeholder retrieval — swap in Module 07's real embedding and cosine similarity code before treating this as production-ready.

8. Common Mistakes

- **Trying to build all 8 projects shallowly** instead of one or two properly — depth teaches more than breadth here.
- **Skipping the architecture sketch** and diving straight into code, leading to costly restructuring later.
- **Underestimating safety needs** (Module 10) for projects with real-world actions — email sending, code changes, automated workflows.
- **Not scoping a minimal v1**, trying to build every feature at once instead of shipping something small and useful first.
- **Ignoring cost estimation** (Module 09) until after a project is already running at scale.

9. Best Practices

- **Pick a project that solves a real need you have** — genuine motivation carries you through the inevitable debugging.
- **Sketch the architecture before coding**, mapping which modules/techniques each part needs.
- **Scope a minimal v1 first**, then add enhancements incrementally.
- **Apply Module 10's safety practices** proportional to the project's real-world consequences.
- **Build your Module 09 evaluation habit** into the project from the start, not as an afterthought.

10. Hands-on Activities

Easy 5 exercises

1. Rank all 8 projects from easiest to hardest for you personally, and explain your top and bottom pick.
2. For your chosen project, list 3 real risks (Module 10) specific to it.
3. Write a one-paragraph "pitch" for your chosen project as if describing it to a user.
4. List the exact API/tool integrations your chosen project will need.
5. Define your project's minimal v1 scope in 3 bullet points.

Intermediate 3 exercises

1. Build the Chat with PDF skeleton from Section 7 with a real PDF and real embeddings (Module 07).
2. Build a minimal version of your chosen project's core loop (even without full UI).
3. Write a small evaluation test set (Module 09) for your project and run it against your v1.

Challenge 2 exercises

1. Take your v1 project and add one genuinely agentic capability (Module 08) to it — a multi-step loop it didn't have before.
2. Add a full guardrail system (Module 10) to your project appropriate to its real-world risk level.

11. Mini Project (Capstone)

Goal

Build one complete project from Section 3, end to end — this is the true capstone of the entire 15-module bucket.

Requirements

- Pick ONE project from Section 3 (Chat with PDF is a great choice if unsure — it exercises the most modules cleanly)
- An Anthropic API key and any other tools your chosen project needs

Step-by-Step Instructions

1. Write a one-paragraph spec: what does this do, for whom, and what's the minimal v1?
2. Sketch the architecture: inputs → processing steps → outputs, noting which modules/techniques each step uses.
3. Build the core loop first (no UI needed) and test it manually with 3-5 realistic inputs.
4. Add structured output validation (Module 04) and appropriate guardrails (Module 10) for your project's risk level.
5. Build a small evaluation test set (Module 09) and confirm your v1 performs acceptably against it.
6. Write a short README documenting what you built and which modules/techniques it uses (bonus: use your own Module 13 documentation-generator skills for this).

Expected Output

```
A working project with:  
- Clear spec and minimal v1 scope  
- Core functionality tested against 3-5 realistic inputs  
- Structured output validation + appropriate guardrails  
- A small evaluation test set with results  
- A README documenting the build
```

Possible Improvements

- Add a v2 feature from your earlier scoping exercise.
- Deploy it somewhere accessible (even a simple local web UI) so others can try it.
- Add cost/latency monitoring (Module 09) if you plan to keep using it regularly.

12. Review Questions

Multiple Choice (10)

1. Chat with PDF is primarily an application of:

- A) Module 11 only B) The RAG pipeline (Module 07) C) Model comparison only D) No prior module

2. Why is a confirmation step critical for an AI email assistant with send access?

- A) It's optional B) Sending is a high-stakes, hard-to-undo action C) It reduces token cost D) It's required by every email provider

3. The AI research assistant leans most heavily on which combination of modules?

- A) Only Module 02 B) Agents, tool use, RAG, and safety (Modules 06-08, 10) C) Only structured outputs D) Only API selection

4. An AI documentation generator relies most on:

- A) Structured output and prompt chaining B) Agent loops only C) Model comparison only D) No prior modules

5. What's recommended before starting to code any of these projects?

- A) Nothing, just start coding B) Sketch the architecture and map needed modules first C) Build all 8 simultaneously D) Skip evaluation entirely

6. A minimal "v1" scope means:

- A) Every possible feature at once B) The smallest version that's genuinely useful C) No features at all D) Skipping testing

7. AI workflow automation projects can be built:

- A) Only with custom code B) With custom code or no-code platforms (Module 13) C) Only visually, never with code D) Without any tool integration

8. A coding assistant's read-only mode (suggests but doesn't apply changes) is recommended because:

- A) It's slower B) It reduces risk before trusting full write access C) It's required by law D) It uses more tokens

9. Why should you pick 1-2 projects to build properly rather than all 8 shallowly?

- A) Depth teaches more than shallow breadth B) It's faster to build 8 C) All 8 are identical D) There's no difference

10. An AI study assistant benefits most from grounding in:

- A) Random unrelated data B) The learner's own course material via RAG C) No context at all D) Only the model's general trained knowledge

Identification (5)

1. ____ is a project that lets users ask questions grounded strictly in an uploaded document's content.

2. ____ is a project combining AI decision-making with multi-step automated actions across services.

3. ____ is a project that autonomously searches, reads, and synthesizes information with citations.

4. ____ is a project generating structured documentation automatically from a codebase.

5. ____ is the foundational project type built primarily from persona and conversation history management.

True or False (5)

1. You should build all 8 projects shallowly rather than 1-2 properly.

2. Chat with PDF is a strong project for practicing the full RAG pipeline end to end.

3. An AI email assistant with send access should never require human confirmation.

4. Sketching an architecture before coding is recommended for these projects.

5. The AI research assistant is generally one of the more demanding projects due to combining agents, tools, and hallucination handling.

Situational (5)

Situational (5)

- 1. You want to practice RAG end to end in the most self-contained way possible. Which project fits best?**
- 2. You're building an AI coding assistant and want to reduce risk before trusting it with write access. What should your v1 do instead?**
- 3. Your documentation generator produces inconsistent formatting across runs. What technique would fix this?**
- 4. You want your email assistant to draft replies but are worried about accidental sends. What guardrail is essential?**
- 5. You're not sure which project to start with. What criterion should guide your choice?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-A, 5-B, 6-B, 7-B, 8-B, 9-A, 10-B

Identification: 1-Chat with PDF, 2-AI Workflow Automation, 3-AI Research Assistant, 4-AI Documentation Generator, 5-AI Chatbot

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Chat with PDF — the cleanest, most self-contained RAG project.
2. A read-only mode that suggests changes without applying them.
3. A strict template (Module 02, 3.9) enforcing consistent structure.
4. A mandatory human confirmation step before any email is actually sent.
5. Pick a project that solves a real need you personally have — genuine motivation matters more than difficulty ranking.

13. Cheat Sheet

🗺️ Project → Module Map

Chatbot: 02,05 · Coding: 06,08,10 · Study: 02,03,07 ·
Research: 06,07,08,10

🗺️ More Mapping

Chat w/ PDF: 05,07,10 · Docs: 03,04,09 · Email: 02,06,10 ·
Workflow: 06,09,10,13

✓ Build Order

Spec → architecture sketch → core loop →
validation/guardrails → evaluation → docs.

🏆 Golden Rule

Depth beats breadth — build 1-2 projects properly, not 8
shallowly.

14. Key Takeaways

- These 8 project types cover the most common real-world AI product categories, each mapping to specific modules from this course.
- AI Chatbot is the foundational starting point; AI Research Assistant is the most demanding, combining agents, tools, RAG, and safety together.
- Chat with PDF is the cleanest single project for mastering the full RAG pipeline end to end.
- Projects with real-world consequences (email sending, code changes, automated workflows) need proportional safety measures (Module 10).
- Sketching architecture and scoping a minimal v1 before coding saves significant rework later.
- Building 1-2 projects properly teaches far more than building 8 shallowly — depth over breadth.

15. Additional Resources

Module Cross-References

- Module 04 — Structured Outputs, Module 05 — Context Engineering, Module 06 — Tool Use, Module 07 — RAG, Module 08 — AI Agents, Module 09 — Prompt Optimization, Module 10 — AI Safety, Module 13 — AI Workflows

Tools

- Anthropic Console for rapid prototyping and testing any of these project ideas before writing full application code.

Communities & Further Learning

- Open-source AI project repositories for inspiration and reference implementations of similar project types.